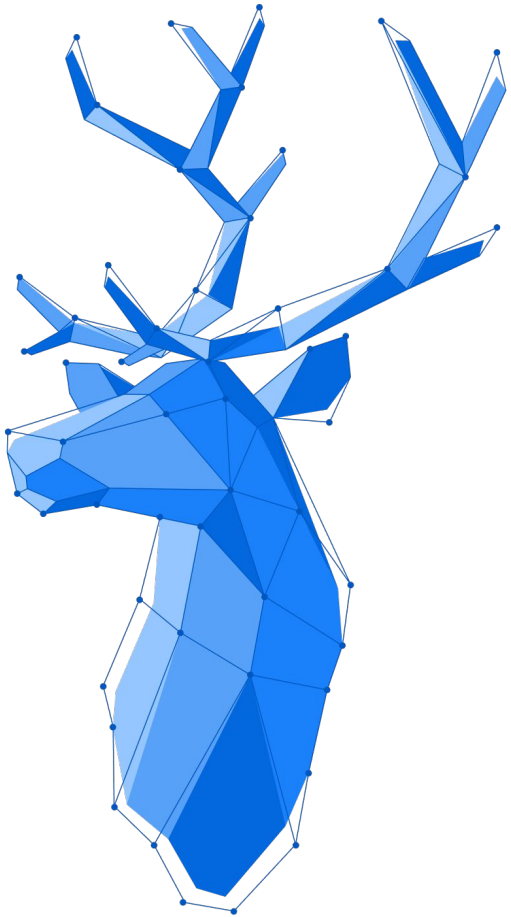
vaadin



Asynchronous Processes

By Gerald Koch

Asynchronous Processes

Preface/Agenda

- All the UI frameworks (Spring, Vaadin, JavaFX, etc.) have something in common when it comes to UI-blocking long-running tasks – i.e. like exports: They execute it in another thread to “unfreeze” the UI. To get back to the UI thread they use a special method:
    - Swing: `Swing.invokeLater(Runnable runnable)`
    - Vaadin: `UI.access(Command command)`
- Additionally, to that we are using Spring which causes some session issues when you do not copy the SecurityContext to other threads.

How to address those topics will be explained in the following:

- Step 1: Long-running tasks in extra thread
- Step 2: Adding Spring context
- Step 3: Involving Vaadin UI
- Step 4: Progress Dialog

Asynchronous Processes

Step 1 - Separate Tasks to a Thread

The `CompletableFuture` is always a good way to go. It works like the following:

If you want to process a result, use the following chain:

```
CompletableFuture
    .supplyAsync(Supplier<U> longRunningTask)
    .whenComplete(BiConsumer<? super T, ? super Throwable> processResult)
```

If you don't want to process a result, then use:

```
CompletableFuture
    .runAsync(Runnable longRunningTask)
    .whenComplete(BiConsumer<Void, ? super Throwable> executeAfter)
```

Attention: `supplyAsync`/`runAsync` as well as `whenComplete` create a new Thread! New threads don't have the Spring context you are using in your UI thread.

Asynchronous Processes

Step 2 - Spring context in threads

The UI Thread of a Spring application contains a security context with information about the authenticated user. The Spring context for non-UI threads is not present, so method calls like `SecurityContextHolder.getContext().getAuthentication().getPrincipal()` will return null or create a NullPointerException.

This information needs to be transferred to the execution thread using the `SecurityContextHolder` methods `getContext()` and `setContext(SecurityContext context)`. Please also take care that the context is reset after you don't need the thread anymore. Ideally you will execute the thread content in a try-finally-construct and do the reset in the finally block.

Asynchronous Processes

Step 2 - Abstract Code Example

```
// save context of UI thread
SecurityContext uiContext = SecurityContextHolder.getContext();
// start new thread
CompletableFuture.runAsync(() -> {
    // save thread context
    SecurityContext context = SecurityContextHolder.getContext();
    // set context of the UI thread
    SecurityContextHolder.setContext(uiContext);
    try {
        // execute thread stuff
    }
    finally {
        // restore thread context
        SecurityContextHolder.setContext(context);
    } // end thread
}).whenComplete((result, error) -> { ... };
```

vaadin

Asynchronous Processes

Step 2 - Optimizations

```
public static void doInContext(SecurityContext outerContext, Runnable runnable){
    SecurityContext threadContext = SecurityContextHolder.getContext();
    SecurityContextHolder.setContext(outerContext);
    try { runnable.run(); } // execute runnable
    finally { SecurityContextHolder.setContext(threadContext); }
 }

public static <T> T doInContext(SecurityContext outerContext, Supplier<T> supplier){
    SecurityContext threadContext = SecurityContextHolder.getContext();
    SecurityContextHolder.setContext(outerContext);
    try { return supplier.get(); } // execute supplier
    finally { SecurityContextHolder.setContext(threadContext); }
 }
```

Asynchronous Processes

Step 3 - Coming back to the UI

When asynchronous Threads are executed, we cannot simply call UI methods. These UI actions like showing success messages and updating progress dialogs need to be executed in UI thread. When UI functionality should be executed from non-UI threads, they need to use special methods to access the UI thread:

- Swing: static call to `Swing.invokeLater(…)`
- Vaadin: non static call to `UI.access(…)` , because Vaadin is a multi-user environment. This means, there are multiple UIs. This method needs to be called on an instance of the UI class.

To finally make it happen that Vaadin triggers a refresh of the UI, we need to enable Push. For this we annotate our class `Application` with `@Push`.

Asynchronous Processes

Step 3 - Code Example

```
class UiDemo extends FlexLayout {
    private final Button button = new Button("Demo", e -> doMultiThreaded());
    public UiDemo() {
        button.setDisableOnClick(true);
        add(button);
    }
    public void doMultiThreaded() {
        SecurityContext uiContext = SecurityContextHolder.getContext();   // save context of UI thread
        Optional<UI> ui = getUi(); //fetch the UI object
        CompletableFuture // setup completable future structure (step 1)
            .supplyAsync(() -> SecurityUtils.doInContext(uiContext, () -> longRunning(...)))
            .whenComplete((result, error) ->
            ui.ifPresent(u -> { //run when UI is attached
                        if(u.isClosing()) { return; } //check if UI is closing or was closed
                        // call the UI.access method and set the button enabled again
                        u.access(() -> button.setEnabled(true));
                    }));
        }
    }
```

vaadin

Asynchronous Processes

Step 3 - Optimizations

```
public static void doInUiThread(Component component, Consumer<UI> command){
    doInUiThread(component.getUI(), command);
}

public static void doInUiThread(UI ui, Consumer<UI> command){
    doInUiThread(Optional.ofNullable(ui), command);
}

public static void doInUiThread(Optional<UI> ui, Consumer<UI> command){
    if(!ui.isPresent() || ui.get().isClosing()){
        return;
    }
    ui.get().access(() -> command.accept(ui.get()));
}
```

Asynchronous Processes

Step 4 - Progress Dialog

- We learned:
  - Creating threads with CompletableFuture
  - Using Spring context in other threads
  - Accessing the UI from non-UI threads

- Progress Dialog:
  - Frequently updates the UI while execution of long-running task
  - Use ProgressListener as communication between task and UI
  - Closing after the long-running task completes or fails
  - Ideally shows the current position (current of goal)
  - **Attention:** too many UI updates might create performance issues

Step 4 - Progress Listener

The ProgressListener interface is used to submit the maximum and the current value of the implementing class.

```
@FunctionalInterface
public interface ProgressListener {
    void setProgress(double max, double current);
}
```

Step 4 - Progress Dialog Properties

- Implements ProgressListener
- Automatically accesses UI on call of setProgress(...)
- Automatically calculates ETA after 10s
- Additional methods
  - Constructor with title
  - Builder style properties
    - Title
    - Unit

Asynchronous Processes

Step 4 - Example Long Running Task

```
class Snail {
    // create a collection to save all the listeners
    private Collection<ProgressListener> listeners = Collections.synchronizedSet(new  LinkedHashSet<>());
    public void creep (int iterations, long sleepTime) {
        for(int i = 0; i < iterations; i++) {
            try {
                Thread.sleep(sleepTime);
                fireProgress(iterations, i + 1); // fire listener
            } catch (Exception ignored) {}
        }
    }
    private void fireProgress(double max, double current) { // iterate over the listeners and fire the values
        new LinkedHashSet<>(listeners).forEach(l -> l.setProgress(max, current);
    }
    public Registration addProgressListener(ProgressListener listener) { // register listener
        // Registration is a class which allows to keep a listener object and remove it to avoid memory leaks
        return Registration.addAndRemove(listeners, listener);
      }
}
```

Asynchronous Processes

Step 4 - Example on the UI Side

```
class UiDemo extends FlexLayout {
    private final Button button = new Button("Demo", e -> doLongRunningTask());
    public UiDemo() {
        button.setDisableOnClick(true);
        add(button);
    }
    public void doSomethingInTheUiFromAnotherThread() {
    ProgressDialog pDialog =new ProgressDialog("Traveling the World").withUnit(" mm").open();
    Snail snail = new Snail();
    snail.addProgressListener(pDialog);
    SecurityContext uiCtxt = SecurityContextHolder.getContext(); // save UI context
    CompletableFuture.supplyAsync(() -> doInContext(uiCtxt, snail.creep(50, 100))
        .whenComplete((result, error) -> doInUiThread(this,() -> {
            button.setEnabled(true);
            pDialog.close();
        }));
    }
}
```

Asynchronous Processes

Step 4 - Final Result